

RemSolution — Plan de développement

État réel du code au commit `139750e` · confronté à votre liste de points · ce qui reste, ce qui doit être amélioré, les problèmes, et les estimations.

0. Résumé exécutif

La majorité de votre liste est déjà développée. En confrontant vos points au code, la répartition est la suivante :

	Points
✓ Déjà fait (et souvent au-delà de ce que la note demandait)	34
● Partiel — la base existe, il manque une partie	9
✕ À faire	14
● Problème / dette à corriger	7

Reste à faire, hors mobile et GPS : 38,5 jours ≈ 8 semaines (détail chiffré en §5). Avec le mobile (deux applications) et le GPS : ~20 semaines.

Trois points bloquent la mise en production et doivent passer en premier (détaillés en §4) : la création automatique de l'administrateur d'agence, les sauvegardes, et le déploiement des migrations. Aucun des trois n'est gros — environ 4 jours au total — mais aucun ne peut être différé.

1. Ce qui est déjà fait

À noter avant de lire : plusieurs points de votre liste sont non seulement faits, mais implémentés plus finement que la note ne le demandait. Inutile de les replanifier.

Administrateur système

- CRUD types de dépenses, marques, modèles, types de services additionnels — **fait**
- CRUD agences — **fait** (avec pin sur carte, devise, succursales créées dans la même transaction)
- CRUD utilisateurs, avec attribution d' `AgencyId` — **fait** (`CreateAgencyUserByAdminCommand` , `ResetAgencyUserPasswordCommand` , activation/désactivation)
- Abonnements d'agences + CRUD formules — **fait** (quotas voitures/clients/utilisateurs appliqués sous verrou, statuts Actif/Suspendu/Expiré)
- Tableau de bord : toutes agences / abonnements / voitures et clients par pays — **fait**, et plus complet que la note : agences au quota, abonnements arrivant à échéance, agences sans abonnement, répartition par formule, recettes des formules actives
- « Toutes les données de chaque agence + possibilité de tout faire pour chacune » — **fait** via l'entrée en espace de travail d'agence (`PlatformAdminImpersonationMiddleware`), chaque entrée étant auditée
- Import de modèles de contrat/facture par défaut — **fait** (`ImportDocumentTemplateCommand` , `DocumentTemplateExamples` avec contrat et facture standard, localisés)

Administrateur / utilisateurs d'agence

- CRUD utilisateurs avec attribution d'autorisations — **fait** (écran Équipe ; n'offre que les autorisations des modules activés)
- Configuration : types de dépenses, types de services additionnels, choix du modèle de facture — **fait** (`SetDefaultDocumentTemplateCommand`)
- CRUD voitures, clients, réservations, locations, paiements — **fait**
- Modèle de contrat avec affectation automatique des champs, et manuelle quand le champ n'est pas clair — **fait**, exactement comme demandé : liaison automatique quand le nom de la variable est un chemin connu, sinon *valeur fixe, demander à chaque fois ou vide*
- Ajout d'une location complète (client existant ou nouveau, voiture, services additionnels, facture, paiements, contrat) — **fait**
- Ajout de dépenses (voiture / type / montant) — **fait**
- Crédits (clients : montant location/payé ; dépenses : montant/payé) — **fait**, et l'interface Dépenses a bien été fusionnée dans Paiements comme vous le demandiez
- Tableau de bord clients/argent par mois, année ou date choisie — **fait**
- Demandes de réservation (accepter / refuser avec motif) — **fait**, motif obligatoire et visible du client
- Messagerie avec le client en cours de location — **fait**, et fermée automatiquement quand la location est clôturée (`ChatMessage.CanPostTo`)
- Widgets d'accueil au choix de l'utilisateur, limités à ses autorisations — **fait**
- Succursales avec adresse choisie sur la carte — **fait**

- Calendrier — **fait** (`GetBookingCalendarQuery` , disponible comme panneau d'accueil)
- Notifications / e-mails (entretien voitures, locations en retard, réservations à venir, pour le personnel et les clients concernés) — **fait**, 6 types, balayage horaire par Hangfire
- Statistiques par mois/année, avec filtre voiture ou toutes voitures — **fait** (granularité mois/année, filtre `CarId` , accès depuis la fiche voiture)
- Depuis l'accueil : accès direct aux réservations à valider, locations en retard, réservations du jour — **fait** (écran Accueil unifié en un seul appel)
- Recadrage de la photo depuis la CIN, affichée dans la liste clients — **fait** (`SkiaPortraitCropper` , détection de peau en YCbCr, repli sur l'emplacement conventionnel)
- Voitures avec photos en liste et en détail — **fait** (original + vignette + moyenne, générées en arrière-plan)
- Fiche voiture : dépenses liées groupées par type — **fait**
- Caution — **fait** (`DepositAmount` sur location et réservation, reportée à la conversion)
- Liste clients/voitures : historique des locations, crédits, louer directement depuis la liste, retour si loué, statut — **fait**
- Paiement direct + justificatif joint au paiement — **fait** (`AddPaymentProofFile`)
- Client : email + regroupement avec les comptes utilisateurs, invitation à l'espace client — **fait** (`InviteClientCommand` , envoi automatique sur création/modification/location)
- Client : notation après la location — **fait** (une note par location terminée, sur la voiture et/ou l'agence)
- Client : choix pays/lieu + dates, vue agence avec sa note et ses voitures — **fait** (+ vue carte avec « chercher dans cette zone »)
- Prolongation de la date de fin sans historique séparé — **fait** (`ChangeRentingEndDateCommand` , retarification)
- Modules et autorisations liés (features → permissions) — **fait**, 16 modules, `FeatureCatalog` fait l'intersection
- « Réservation : pourquoi les actions de l'agence sont dans l'entité » — **résolu** : `Reservation` est un vrai agrégat avec machine à états gardée, setters privés et événements de domaine ; l'identité de l'appelant est vérifiée par la politique d'autorisation, pas par la structure de l'entité

2. Partiel — la base existe, il manque une partie

2.1 Tests — 🌀 (déséquilibrés, pas absents) · 5 j

619 tests existent : 479 fonctionnels, 119 applicatifs, **21 seulement au niveau domaine**. Les projets `Infrastructure.IntegrationTests` et `Web.AcceptanceTests` sont des coquilles vides (9 fichiers, 0 test).

Le déséquilibre n'est pas un oubli de rigueur : il n'y a presque rien à tester au niveau domaine parce que `Renting` est une entité anémique (voir §4.4). Rendre `Renting` riche fait apparaître les tests domaine naturellement.

- Tests domaine après enrichissement de `Renting` : **2 j**
- Un parcours E2E fiable (SpecFlow + Playwright déjà câblés) : **1,5 j**
- Tests d'intégration infrastructure (stockage fichiers, imagerie, jobs) : **1,5 j**

2.2 Interface de location — 🌀 (fonctionnelle, à retravailler) · 3 j

Vous notiez : « *interface de location pas claire et pas belle, le prix devrait être calculé automatiquement par date mais modifiable, un assistant serait peut-être mieux* ».

Le calcul automatique existe (`IPricingService` calcule depuis les dates et le tarif journalier, le prix est figé à la réservation et modifiable). L'assistant, non : `renting-form.component` reste un formulaire unique. À faire : un assistant en étapes (client → voiture et dates → services additionnels → prix → confirmation).

2.3 Paiement en ligne — 🌀 (structure prête, aucun prestataire) · 5 j

Le module `OnlinePayment` existe, l'état `Payée` existe, `PaymentMethod` couvre espèces/carte/virement/chèque. **Aucun prestataire n'est intégré** — pas de Stripe, Paymee, Konnect ou Flouci dans le code. Il faut choisir un prestataire (pour la Tunisie : Paymee ou Konnect), gérer le retour de paiement et les webhooks (voir §4.3 : l'Outbox devient nécessaire ici).

2.4 Hébergement — 🌀 (Bicep existe, non déployé) · 3 j

`infra/` contient des modèles Bicep et `azure.yaml` pour Azure Developer CLI, et un workflow `azure-dev.yml` . Ce qui manque : un déploiement réellement effectué et vérifié, les secrets en Key Vault (l'abstraction existe), le nom de domaine, les certificats.

2.5 CI/CD — 🌀 · 1,5 j

Un seul workflow GitHub (`azure-dev.yml` , déploiement). Il manque : compilation + tests sur chaque PR, tests fonctionnels sur la branche principale, et surtout **l'application des migrations au déploiement** (voir §4.2).

2.6 Sécurité — 🌀 (les fondations sont bonnes, il reste du nettoyage) · 2 j

Solide : isolation par locataire vérifiée par test de convention, autorisations par politique, jetons courts + rafraîchissement, verrou applicatif, RowVersion, audit.

À corriger : mot de passe d’amorçage en dur (`"PlatformAdmin1!"`), fichiers de débogage commités (`cookies_reg.txt` , `info.docx` à la racine, trois `cookies_*.txt` dans ClientApp), limitation de débit et durcissement des points d’entrée publics (téléversements : taille, type réel, extensions), scan de secrets sur l’historique complet du dépôt.

2.7 Signature électronique — 🌀 (à clarifier) · 6 j si réelle

Le libellé français du module dit « Contrats + signature électronique », mais le code ne fait que des **lignes réglées pour signature manuscrite** (`DocumentBlockType.Signatures`). Deux options :

- **Option A — corriger le libellé** (0,2 j). Si la signature papier au comptoir suffit, c’est le choix honnête.
- **Option B — vraie signature électronique** (6 j) : capture de la signature sur écran/tablette, apposition dans le PDF, horodatage, preuve de consentement. Attention : la valeur juridique en Tunisie relève de l’ANCE ; une signature « dessinée » n’est pas une signature électronique qualifiée. À arbitrer avec un juriste avant de développer.

2.8 Commercialisation — 🌀 · hors développement

Rien dans le code, et c’est normal : site vitrine, tarification publique, inscription des agences en libre-service, facturation des abonnements. À noter : **l’inscription en libre-service d’une agence est impossible tant que §4.1 n’est pas corrigé.**

2.9 Statistiques — 🌀 (l’essentiel est là) · 1 j

Il manque l’export (Excel/PDF), régulièrement demandé dès qu’un gérant veut transmettre ses chiffres à son comptable.

3. À faire

3.1 Seuils de dépense par voiture — ✕ · 2,5 j

Votre point : « *depuis le type de dépense, je veux lier ceux à notifier avec les voitures, en création/modification/détail voiture, et le paramètre doit être modifiable. Par exemple : vidange liée à la voiture 15412154 tous les 8000 km et à la voiture 1254548 tous les 10000 km.* »

Aujourd’hui le seuil est porté par `ExpenseType` seul (`AfterMonth` / `AfterKilometer`), donc global à l’agence : une vidange tous les 8000 km pour tout le monde. Un utilitaire ne s’entretient pas comme une citadine — c’est une vraie limite.

À développer : une table de liaison `CarExpenseSchedule(CarId, ExpenseTypeId, AfterMonth?, AfterKilometer?)` , le seuil du type servant de valeur par défaut quand aucune ligne n’existe. Puis le calculateur d’échéances doit consulter la surcharge, et l’interface doit permettre l’édition depuis la fiche voiture **et** depuis le type de dépense (les deux sens que vous décrivez).

3.2 Frais supplémentaires au retour — ✕ · 2 j

Votre point : « *au retour de location, on peut ajouter de l’argent (retard, réparation, kilométrage supplémentaire...)* ».

La boîte de dialogue de retour ne comporte que le kilométrage et la date. On peut contourner par un service additionnel, mais ce n’est pas la bonne place : ce sont des frais constatés au retour, pas des options vendues au départ. À développer : des lignes de frais typées (retard / dommage / kilométrage excédentaire / carburant / nettoyage) saisies dans la boîte de retour et intégrées au prix final et à la facture.

3.3 Application mobile administrateur/agence — ✕ · 5 semaines

Rien n’existe. Décision à prendre : application unique multi-rôles ou deux applications ; et technologie (le plus cohérent avec une équipe .NET : .NET MAUI ou Blazor Hybrid ; le plus rapide à livrer si l’équipe est plutôt web : Ionic/Capacitor par-dessus l’Angular existant, ce qui permet de réutiliser le client NSwag).

Inclut l’OCR des documents pour préremplir le contrat, prévu de longue date — les originaux sont déjà conservés en pleine résolution précisément pour cela.

3.4 Application mobile client — ✕ · 4 semaines

Recherche, réservation, téléversement des documents, messagerie, notifications *push*. À noter : les notifications *push* n’existent nulle part aujourd’hui (tout passe par courriel et notifications dans l’application) — il faut donc ajouter Firebase ou équivalent, ce qui se greffe naturellement sur l’Outbox (§4.3).

3.5 Extension GPS — ✕ · 3 semaines

Rien n’existe côté suivi. `NetTopologySuite` est déjà en place pour la géographie (succursales, recherche par distance), ce qui aide, mais le suivi de véhicule est un autre métier : choix du boîtier, ingestion des positions, historique des trajets, géorepérage, et une décision de volumétrie (une position par minute et par véhicule, c’est des millions de lignes — cela ne va pas dans la base transactionnelle sans réflexion).

3.6 Divers — X

- **Purge des données personnelles et droit à l'effacement** (obligation légale dès qu'on détient des passeports) : politique de rétention, job Hangfire de purge, anonymisation du client en conservant les lignes financières — **1,5 j**
- **Test de charge du chemin de recherche** au volume cible (2 000 agences / 100 000 voitures) pour valider les plans d'exécution des anti-jointures et de la recherche spatiale — **1 j**
- **Signalements (réclamations)** avec écran de triage pour l'administrateur plateforme — **1 j**
- **Export des statistiques** (voir §2.9) — **1 j**

4. Problèmes — par ordre de gravité

4.1 ● BLOQUANT — Aucun administrateur n'est créé avec l'agence

Votre note le dit : « *quand on ajoute une agence, un admin d'agence est créé automatiquement* ». **Ce n'est pas le cas.**

`CreateAgencyCommandHandler` crée l'agence, ses réglages et ses succursales, mais **aucune commande du système n'attribue le rôle `AgencyAdministrator`**. Une agence créée n'a personne qui puisse s'y connecter.

Conséquences : impossible d'ouvrir une agence pour un vrai client sans intervention manuelle en base ; impossible d'ouvrir l'inscription en libre-service.

À faire : créer l'utilisateur administrateur dans la même transaction que l'agence, avec un mot de passe temporaire, l'obligation de le changer à la première connexion (le middleware `PasswordChangeRequiredMiddleware` existe déjà) et l'envoi du courriel d'accueil.

Estimation : 1 j

4.2 ● BLOQUANT — Migrations appliquées au démarrage de l'application

`ApplicationDbContextInitialiser.InitialiseAsync` appelle `MigrateAsync()` au démarrage. Sur une seule instance, cela fonctionne. **Dès la deuxième instance (ou un redémarrage pendant un déploiement), deux processus appliquent les migrations en même temps** — et le résultat va de l'erreur bruyante à un schéma à moitié migré.

Aggravant en multi-locataire : ce n'est pas un client qui tombe, c'est tout le monde.

À faire : générer un *bundle* de migrations (`dotnet ef migrations bundle`) appliqué à l'étape de déploiement, et retirer l'appel au démarrage. **Estimation : 1 j**

4.3 ● BLOQUANT — Aucune sauvegarde, aucune restauration répétée

Aucune trace de sauvegarde dans `infra/`. Sur une base partagée par toutes les agences, **une base perdue, c'est tous les clients perdus**. Et une sauvegarde jamais restaurée n'est pas une sauvegarde : c'est une hypothèse.

À faire : sauvegardes automatiques SQL Server + stockage des fichiers, plus **une restauration réellement effectuée** dans un environnement propre, documentée, avec le temps qu'elle a pris. **Estimation : 2 j**

4.4 ● Renting est une entité anémique alors que Reservation est un agrégat

`Reservation` a une fabrique, des setters privés, des transitions gardées et des événements. `Renting` a des setters publics, aucune fabrique, aucune machine à états — les invariants (fin > début, prix ≥ 0, kilométrage de retour ≥ kilométrage de départ, transitions légales) vivent donc dans les gestionnaires, où ils peuvent être oubliés par le prochain point d'entrée qui touche une location.

C'est aussi la cause directe des 21 tests domaine seulement (§2.1) : il n'y a rien à assérer.

À faire : `Renting.Create(...)`, `Start()`, `Complete(endMileage)`, `Cancel(reason)` avec transitions gardées et événements de domaine ; les gestionnaires se contentent de charger → appeler → sauvegarder. **Estimation : 2,5 j** (+ les 2 j de tests domaine du §2.1)

4.5 ● Le verrou d'écriture est au niveau agence, pas au niveau véhicule

`AcquireTenantWriteLockAsync` verrouille `agency-writes-{agencyId}`. La correction est assurée (c'est un verrou plus large que nécessaire), mais **toutes les écritures d'une agence se sérialisent**, y compris deux réservations sur deux voitures différentes par deux collaborateurs différents.

Invisible à 3 utilisateurs, gênant à 15, bloquant le jour d'un pic. À reprendre en verrou par voiture pour les chemins de disponibilité, en gardant le verrou agence pour ce qui touche réellement les quotas. **Estimation : 1,5 j**

4.6 ● Aucun Outbox — et trois fonctionnalités à venir en ont besoin

Les événements de domaine sont distribués dans la transaction. Correct aujourd'hui. Mais **paiement en ligne (§2.3), notifications push (§3.4) et tout webhook** sont des effets hors processus : sans Outbox, un `commit` suivi d'un plantage perd l'événement silencieusement — un paiement encaissé chez le prestataire et jamais enregistré chez vous.

À faire avant, et non pendant, l'intégration du paiement : table `Outbox` écrite dans la même transaction, distributeur Hangfire. **Estimation : 2 j**

4.7 🌐 Nettoyage

Mot de passe d'amorçage en dur ; fichiers de débogage commités (`cookies_reg.txt` , `info.docx` , trois `cookies_*.txt`) — à supprimer **et** à vérifier qu'ils ne contiennent pas de jetons de session valides, l'historique Git les conservant ; restes du gabarit (`Colour.cs` , `UnsupportedColourException.cs`) ; `docs/PROJECT_OVERVIEW.md` décrivait un état obsolète (corrigé). **Estimation : 0,5 j**

5. Ordre recommandé et estimations

Lot 1 — Débloquer la production (4 j)

Rien ne peut être livré à un vrai client avant.

	Tâche	Effort
4.1	Création automatique de l'administrateur d'agence	1 j
4.2	Migrations au déploiement (bundle)	1 j
4.3	Sauvegardes + restauration répétée	2 j

Lot 2 — Solidifier le domaine (6 j)

À faire avant d'ajouter des fonctionnalités sur les locations, sinon la dette se démultiplie.

	Tâche	Effort
4.4	<code>Renting</code> en agrégat riche	2,5 j
2.1	Tests domaine	2 j
4.5	Verrou par véhicule	1,5 j

Lot 3 — Vos manques fonctionnels (7,5 j)

Les vrais trous de votre liste.

	Tâche	Effort
3.1	Seuils de dépense par voiture	2,5 j
3.2	Frais supplémentaires au retour	2 j
2.2	Assistant de création de location	3 j

Lot 4 — Exploitation et mise en ligne (8,5 j)

	Tâche	Effort
2.6	Durcissement sécurité + nettoyage (avec 4.7)	2,5 j
2.5	CI/CD complet	1,5 j
2.4	Hébergement déployé et vérifié	3 j
3.6	Purge données personnelles (légal)	1,5 j

Lot 5 — Monétisation (8 j)

	Tâche	Effort
4.6	Outbox (prérequis)	2 j
2.3	Paiement en ligne (prestataire tunisien)	5 j
2.9	Export des statistiques	1 j

Lot 6 — Finitions (4,5 j)

	Tâche	Effort
--	-------	--------

2.1	E2E + tests d'intégration	3 j
3.6	Test de charge du chemin de recherche	1 j
2.7	Corriger le libellé signature (option A)	0,5 j

Lot 7 — Extensions (optionnel, à décider)

	Tâche	Effort
3.3	Application mobile agence (+ OCR)	5 sem.
3.4	Application mobile client (+ push)	4 sem.
3.5	Extension GPS	3 sem.
2.7	Vraie signature électronique (option B)	6 j

Total

Périmètre	Durée
Lots 1 à 6 — produit vendable et exploitable	38,5 j ≈ 7,7 semaines
+ Lot 7 complet	~20 semaines

Estimations pour un seul développeur à temps plein, hors mobile où une compétence dédiée change fortement la donne.

6. Décisions à arbitrer avant de coder

Cinq points ne sont pas des tâches mais des choix qui déterminent le travail :

- 1. **Signature électronique** — libellé à corriger, ou vraie fonctionnalité juridiquement valable ? Écart : 0,5 j contre 6 j, plus un avis juridique.
- 2. **Prestataire de paiement** — Paymee, Konnect, Flouci ? Détermine l’effort réel du §2.3 et si le paiement en ligne est même viable pour vos agences.
- 3. **Mobile : technologie** — MAUI (cohérent avec .NET) ou Capacitor par-dessus l’Angular existant (bien plus rapide, réutilise le client NSwag) ? Écart possible : plusieurs semaines.
- 4. **Mobile : une application ou deux ?** Votre liste en prévoit deux. Une seule application multi-rôles coûte moins cher mais mélange deux publics très différents dans les magasins d’applications.
- 5. **GPS : volumétrie et stockage** — à décider avant la première position écrite, pas après. Une base transactionnelle multi-locataire n’est pas l’endroit pour un flux de télémétrie.

7. Ce que je recommande de faire cette semaine

Le Lot 1 en entier, dans l’ordre : administrateur d’agence automatique (1 j), migrations au déploiement (1 j), sauvegardes et restauration répétée (2 j).

Ces quatre jours transforment un projet « presque fini mais non livrable » en projet livrable. Tout le reste peut attendre ; ces trois-là, non.